Mobile Robotics (MCT-454L) - Lab 4 Report

1. Objective

The primary objective of this laboratory session was to transition from individual node execution to system-level management using advanced ROS 2 tools. This involved mastering the creation of ROS 2 launch files to start multiple nodes and services simultaneously, utilizing rosbag2 to record and replay vital topic data for offline analysis, and leveraging the rqt framework to visualize real-time control commands and node graphs.

2. Methodology and Implementation

2.1 Task 1: ROS 2 Launch Files and Simultaneous Execution

Instead of manually initiating the turtlesim environment and spawning additional entities across multiple terminals, a custom launch package (my_launch_pkg) was developed. A Python-based launch file, turtlesim_launch.py, was created to orchestrate the entire startup sequence.

Commands Executed:
ros2 pkg create my_launch_pkg --build-type ament_python --dependencies turtlesim rclpy
colcon build
source install/setup.bash
ros2 launch my_launch_pkg turtlesim_launch.py

Observation: The launch script successfully executed the turtlesim_node and automatically invoked the /spawn service using the ExecuteProcess action, instantly generating turtle2 upon startup. This effectively demonstrated how complex robotic systems can be initialized reliably with a single command.

2.2 Task 2: Data Logging with Rosbag

To simulate real-world data collection, the rosbag2 utility was utilized to capture the kinematic flight data of the robots. While driving turtle1 via keyboard teleoperation, active topic streams were recorded and saved into a local database directory.

Commands Executed:
mkdir -p ~/Rayan_ros2_ws/bags
cd ~/Rayan_ros2_ws/bags
ros2 bag record /turtle1/cmd_vel /turtle2/cmd_vel /turtle1/pose /turtle2/pose

Observation: The system successfully logged all spatial and velocity messages. This bag file acts as a flight recorder, enabling offline trajectory analysis, debugging, and the ability to replay the exact movements of the robots without requiring live manual input.

2.3 Task 3: Follow-the-Leader Behavior (Swarm Logic)

A custom autonomous tracking node (follower.py) was developed to implement a follow-the-leader swarm behavior, utilizing proportional control logic.
Commands Executed:
ros2 run my_launch_pkg follower

Implementation Details & Mathematics:

- **Dual Subscription:** The node subscribed to the pose topics of both turtle1 and turtle2 simultaneously to acquire real-time coordinate feedback.
- **Error Calculation:** The node calculated the Euclidean distance error mathematically as d = sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2} and the heading angle required to face the target as theta = atan2(Delta y, Delta x).
- **Proportional Control:** Proportional gains were applied to generate real-time Twist commands. As turtle1 moved, turtle2 dynamically altered its linear and angular velocity to hunt down the leader, autonomously decelerating and stopping when the distance constraint (d> 0.5 units) was breached to avoid collisions.

2.4 Task 4: Signal Visualization with rqt_plot

To verify the integrity and smoothness of the control signals, the graphical rqt_plot interface was utilized.

Commands Executed:
rqt

Observation: By plotting the /turtle1/cmd_vel/linear/x and /turtle1/cmd_vel/angular/z sub-topics simultaneously, the exact spikes and curves of the teleoperation commands were graphically visualized in real-time. The generated plots provided conclusive proof of smooth, instantaneous command transmission from the keyboard inputs across the ROS 2 network.

3. Conclusion

This lab successfully introduced critical infrastructure tools utilized in professional robotics engineering. By mastering automated Launch files, persistent data recording with Rosbag, and live kinematic visualization with rqt, alongside writing an advanced multi-robot tracking algorithm, a robust foundation for managing, deploying, and debugging complex autonomous systems has been established.